

8η Εργαστηριακή Άσκηση (12.12.2025)

Ομάδα 60	AM
Διονύσιος Γεώργιος Μουρελάτος	

Συναρτήσεις

I. PCA9555, TWI, LCD

```
#define F_CPU 16000000UL
#define PD3 3
#define PD4 4

#include<avr/io.h>
#include<avr/interrupt.h>
#include<util/delay.h>
#include<stdio.h>
#include<string.h>

#define PCA9555_0_ADDRESS 0x40
#define TWI_READ 1
#define TWI_WRITE 0
#define SCL_CLOCK 100000L

//A0=A1=A2=0 by hardware
// reading from twi device
// writing to twi device
// twi clock in Hz
//FscL=Fcpu/(16+2*TWBR0_VALUE*PRESCALER_VALUE)
#define TWBR0_VALUE ((F_CPU/SCL_CLOCK)-16)/2

// PCA9555 REGISTERS
typedef enum {
    REG_INPUT_0 = 0,
    REG_INPUT_1 = 1,
    REG_OUTPUT_0 = 2,
    REG_OUTPUT_1 = 3,
    REG_POLARITY_INV_0 = 4,
    REG_POLARITY_INV_1 = 5,
    REG_CONFIGURATION_0 = 6,
    REG_CONFIGURATION_1 = 7
} PCA9555_REGISTERS;
```

```

//----- Master Transmitter/Receiver -----
#define TW_START          0x08
#define TW_REP_START      0x10

//----- Master Transmitter -----
#define TW_MT_SLA_ACK      0x18
#define TW_MT_SLA_NACK     0x20
#define TW_MT_DATA_ACK     0x28

//----- Master Receiver -----
#define TW_MR_SLA_ACK      0x40
#define TW_MR_SLA_NACK     0x48
#define TW_MR_DATA_NACK    0x58

#define TW_STATUS_MASK     0b11111000
#define TW_STATUS (TWSR0 & TW_STATUS_MASK)

//initialize TWI clock
void twi_init(void)
{
    TWSR0 = 0;
    // PRESCALER_VALUE=1
    TWBR0 = TWBR0_VALUE; // SCL_CLOCK 100KHz
}

// Read one byte from the twi device (request more data from device)
unsigned char twi_readAck(void)
{
    TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWEA);
    while(!(TWCR0 & (1<<TWINT)));
    return TWDRO;
}

//Read one byte from the twi device, read is followed by a stop condition
unsigned char twi_readNak(void)
{
    TWCR0 = (1<<TWINT) | (1<<TWEN);
    while(!(TWCR0 & (1<<TWINT)));
    return TWDRO;
}

// Issues a start condition and sends address and transfer direction.
// return 0 = device accessible, 1= failed to access device
unsigned char twi_start(unsigned char address)
{
    uint8_t twi_status;
    // send START condition
    TWCR0 = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);
    // wait until transmission completed

```

```

while(!(TWCR0 & (1<<TWINT)));
// check value of TWI Status Register.
twi_status = TW_STATUS & 0xF8;
if ( (twi_status != TW_START) && (twi_status != TW_REP_START)) return 1;
// send device address
TWDR0 = address;
TWCR0 = (1<<TWINT) | (1<<TWEN);
// wait until transmission completed and ACK/NACK has been received
while(!(TWCR0 & (1<<TWINT)));
// check value of TWI Status Register.
twi_status = TW_STATUS & 0xF8;
if ( (twi_status != TW_MT_SLA_ACK) && (twi_status != TW_MR_SLA_ACK) )
{
    return 1;
}
return 0;
}

// Send start condition, address, transfer direction.
// Use ack polling to wait until device is ready
void twi_start_wait(unsigned char address)
{
    uint8_t twi_status;

    while ( 1 )
    {
        // send START condition
        TWCR0 = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);

        // wait until transmission completed
        while(!(TWCR0 & (1<<TWINT)));

        // check value of TWI Status Register.
        twi_status = TW_STATUS & 0xF8;
        if ( (twi_status != TW_START) && (twi_status != TW_REP_START))
continue;

        // send device address
        TWDR0 = address;
        TWCR0 = (1<<TWINT) | (1<<TWEN);

        // wait until transmission completed
        while(!(TWCR0 & (1<<TWINT)));

        // check value of TWI Status Register.
        twi_status = TW_STATUS & 0xF8;
        if ( (twi_status == TW_MT_SLA_NACK) || (twi_status == TW_MR_DATA_NACK)
)
        {
            /* device busy, send stop condition to terminate write operation

```

```

*/
    TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);

    // wait until stop condition is executed and bus released
    while(TWCR0 & (1<<TWSTO));

    continue;
}
break;
}
}

// Send one byte to twi device, Return 0 if write successful or 1 if write
failed
unsigned char twi_write( unsigned char data )
{
    // send data to the previously addressed device
    TWDR0 = data;
    TWCR0 = (1<<TWINT) | (1<<TWEN);
    // wait until transmission completed
    while(!(TWCR0 & (1<<TWINT)));
    if( (TW_STATUS & 0xF8) != TW_MT_DATA_ACK) return 1;
    return 0;
}

// Send repeated start condition, address, transfer direction
//Return: 0 device accessible
//1 failed to access device
unsigned char twi_rep_start(unsigned char address)
{
    return twi_start( address );
}

// Terminates the data transfer and releases the twi bus
void twi_stop(void)
{
    // send stop condition
    TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);
    // wait until stop condition is executed and bus released
    while(TWCR0 & (1<<TWSTO));
}

void PCA9555_0_write(PCA9555_REGISTERS reg, uint8_t value)
{
    twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
    twi_write(reg);
    twi_write(value);
    twi_stop();
}

```

```

uint8_t PCA9555_0_read(PCA9555_REGISTERS reg)
{
    uint8_t ret_val;
    twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
    twi_write(reg);
    twi_rep_start(PCA9555_0_ADDRESS + TWI_READ);
    ret_val = twi_readNak();
    twi_stop();
    return ret_val;
}

void write_2_nibbles(uint8_t lcd_data) {
    uint8_t temp;

    // high nibble
    temp = (PCA9555_0_read(REG_OUTPUT_0) & 0x0F) | (lcd_data & 0xF0); //
    Keep lower 4 bits of PIND and set high nibble of lcd_data
    PCA9555_0_write(REG_OUTPUT_0 , temp); //
    Output the high nibble to PORTD
    PCA9555_0_write(REG_OUTPUT_0, PCA9555_0_read(REG_OUTPUT_0) | (1 << PD3));
    // Enable pulse high
    _delay_us(1);
    PCA9555_0_write(REG_OUTPUT_0, PCA9555_0_read(REG_OUTPUT_0) & ~(1 <<
    PD3)); // Enable pulse low

    // low nibble
    lcd_data <<= 4; // Move low nibble to high
    nibble position
    temp = (PCA9555_0_read(REG_OUTPUT_0) & 0x0F) | (lcd_data & 0xF0); //
    Keep lower 4 bits of PIND and set high nibble of new lcd_data
    PCA9555_0_write(REG_OUTPUT_0 , temp); //
    Output the low nibble to PORTD
    PCA9555_0_write(REG_OUTPUT_0 , PCA9555_0_read(REG_OUTPUT_0) | (1 <<
    PD3)); // Enable pulse high
    _delay_us(1);
    PCA9555_0_write(REG_OUTPUT_0, PCA9555_0_read(REG_OUTPUT_0) & ~(1 <<
    PD3)); // Enable pulse low
}

void lcd_data(uint8_t data)
{
    PCA9555_0_write(REG_OUTPUT_0, PCA9555_0_read(REG_OUTPUT_0) | 0x04);
    // LCD_RS = 1, (PD2 = 1) -> For Data
    write_2_nibbles(data); // Send data
    _delay_ms(5);
    return;
}

void lcd_command(uint8_t data)

```

```

{
    PCA9555_0_write(REG_OUTPUT_0, PCA9555_0_read(REG_OUTPUT_0) & 0xFB);
// LCD_RS = 0, (PD2 = 0) -> For Instruction
    write_2_nibbles(data);
    _delay_ms(5);
    return;
}

void lcd_clear_display()
{
    lcd_command(0x01);
    _delay_ms(5);
    return;
}

void lcd_init() {
    _delay_ms(200);

    // 0x30 for 8-bit mode (x3)
    PCA9555_0_write(REG_OUTPUT_0, 0x30); // #1
    PCA9555_0_write(REG_OUTPUT_0, PCA9555_0_read(REG_OUTPUT_0) | (1 << PD3));
// Enable pulse
    _delay_us(1);
    PCA9555_0_write(REG_OUTPUT_0, PCA9555_0_read(REG_OUTPUT_0) & ~(1 <<
PD3)); // Clear enable
    _delay_us(250);

    PCA9555_0_write(REG_OUTPUT_0, 0x30); // #2
    PCA9555_0_write(REG_OUTPUT_0, PCA9555_0_read(REG_OUTPUT_0) | (1 << PD3));
    _delay_us(1);
    PCA9555_0_write(REG_OUTPUT_0, PCA9555_0_read(REG_OUTPUT_0) & ~(1 <<
PD3));
    _delay_us(250);

    PCA9555_0_write(REG_OUTPUT_0, 0x30); // #3
    PCA9555_0_write(REG_OUTPUT_0, PCA9555_0_read(REG_OUTPUT_0) | (1 << PD3));
    _delay_us(1);
    PCA9555_0_write(REG_OUTPUT_0, PCA9555_0_read(REG_OUTPUT_0) & ~(1 <<
PD3));
    _delay_us(250);

    // Send 0x20 command to switch to 4-bit mode
    PCA9555_0_write(REG_OUTPUT_0, 0x20);
    PCA9555_0_write(REG_OUTPUT_0, PCA9555_0_read(REG_OUTPUT_0) | (1 << PD3));
    _delay_us(1);
    PCA9555_0_write(REG_OUTPUT_0, PCA9555_0_read(REG_OUTPUT_0) & ~(1 <<
PD3));
    _delay_us(250);

    // Set 4-bit mode, 2 lines, 5x8 dots

```

```

    lcd_command(0x28);

    // Display, Cursor = ON, OFF
    lcd_command(0x0C);

    // Clear display
    lcd_clear_display();

    // Increase address, no display shift
    lcd_command(0x06);
}

void lcd_string(const char *str) {
    while (*str) lcd_data(*str++);
}

void clear_2nd_line() {
    lcd_command(0xC0); // Move to start of line 2
    lcd_string("                "); // 16 spaces (assuming 16x2 LCD)
}

void print_lcd_with_number(int n, const char *text) {
    clear_2nd_line();
    lcd_command(0xC0);
    char buffer[16];
    snprintf(buffer, sizeof(buffer), "%d.%s", n, text);
    lcd_string(buffer);
}

```

II. ADC, PWM

```

void PWM_init()
{
    // Initialize TMR1A in fast PWM 8-bit mode with non-inverted output
    // Prescaler = 1, to get 62.5kHz waveform in PB1
    TCCR1A = (1 << WGM10) | (1 << COM1A1);
    TCCR1B = (1 << CS10) | (1 << WGM12);
}

void ADC_init()
{
    /* Channel (ADC0) to read from POT1 -> MUX all nulls
    * For voltage reference selection: REFS0 = 1, REFS1 = 0
    * Right adjustment: ADLAR = 0
    * ADC0: MUX3 = 0, MUX2 = 0, MUX1 = 0, MUX0 = 0
    */
    ADMUX = (1 << REFS0);
    // Same as the above ADMUX = 0b01000000;
}

```

```

/* Enable ADC: ADEN = 1
 * No conversion from analog to digital yet: ADSC = 0
 * Disable ADC interrupt: ADIE = 0
 * Prescaler: f_ADC = 16MHz / prescaler and
 * 50kHz <= f_ADC <= 200kHz for 10-bits accuracy. So,
 * division factor = 128 -> gives f_ADC = 125kHz
 */
ADCSRA = (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0); ;
// Same as the above ADCSRA = 0b10000111;
}

uint16_t ADC_conv()
{
    ADCSRA |= (1 << ADSC);          // Start conversion
    while (ADCSRA & (1 << ADSC)); // Wait until done
    return ADC;                     // Return 10-bit result
}

```

III. One Wire, Temperature

```

/* === ONE WIRE FUNCTIONS + TEMPERATURE === */

/* Detects if there is a connected device */
int one_wire_reset () {

    DDRD |= (1 << PD4);          // set PD4 as output

    PORTD &= ~(1 << PD4);        // set 0
    _delay_us(480);              // wait 480 us

    DDRD &= ~(1 << PD4);        // set PD4 as input
    PORTD &= ~(1 << PD4);        // disable pull-up

    _delay_us(100);              // wait 100us for connected devices
    // to transmit the presence pulse

    uint8_t x = PIND & (1 << PD4); // read PORTD

    _delay_us(380);              // wait for 380us

    if (x == 0x10)
        return 0; // if there is NO device (PD4 == 1)
    else
        return 1; // if there is a connected device (PD4 == 0)

}

/* Reads one BIT from the sensor via PD4 */

```



```

uint8_t one_wire_receive_bit () {

    DDRD |= (1 << PD4);          // set PD4 as output
    PORTD &= ~(1 << PD4);
    _delay_us(2);

    DDRD &= ~(1 << PD4);        // set PD4 as input
    PORTD &= ~(1 << PD4);        // disable pull-up
    _delay_us(10);               // wait for 10us

    /* if PD4 set, return 1, else 0 */
    uint8_t bit = (PIND & (1 << PD4)) ? 1 : 0;
    _delay_us(49);               // delay 49us to meet the standards

    return bit;
}

/* Reads one BYTE from the sensor */
uint8_t one_wire_receive_byte () {

    uint8_t byte = 0x00;
    uint8_t bit = 0x00;

    /* Received LSB to MSB */
    for (int i = 0; i < 8; i++) {
        bit = one_wire_receive_bit();
        byte |= (bit << i); // 'or' to insert bit into byte
    }

    return byte;
}

/* Transmits one BIT to the sensor via PD4 */
void one_wire_transmit_bit (uint8_t bit) {

    DDRD |= (1 << PD4);          // set PD4 as output
    PORTD &= ~(1 << PD4);        // set PD4 to 0
    _delay_us(2);

    /* Transmit 1 or 0 based on 'bit' */
    if (bit & 0x01)
        PORTD |= (1 << PD4);
    else
        PORTD &= ~(1 << PD4);

    _delay_us(58);               // wait 58us for device to sample the
line

```

```

    DDRD &= ~(1 << PD4);          // set PD4 as input
    PORTD &= ~(1 << PD4);          // disable pull-up

    _delay_us(1);                  // recovery time 1us
}

/* Transmits one BYTE to the sensor */
void one_wire_transmit_byte (uint8_t byte) {

    uint8_t bit;

    /* Transmit one bit at a time, using masking */
    for (int i = 0; i < 8; i++) {
        bit = (0x01 & (byte >> i));
        one_wire_transmit_bit(bit);
    }
}

uint16_t receive_temperature () {

    if (!(one_wire_reset()))
        return 0x800; // error: no connected device

    one_wire_transmit_byte(0xCC); // bypass device selection
    one_wire_transmit_byte(0x44); // start temperature sensing

    /* Block until temperature is ready (1) */
    while (!(one_wire_receive_bit()));

    if (!(one_wire_reset())) // re-initialize
        return 0x800;        // just for safety

    one_wire_transmit_byte(0xCC);
    one_wire_transmit_byte(0xBE); // 16-bit temp reading

    uint16_t temperature = 0x00;
    temperature |= one_wire_receive_byte(); // read the
LSByte (0x0X)
    temperature |= ((uint16_t)one_wire_receive_byte() << 8); // insert
the MSByte by shifting left 8 times (0xYX)

    return temperature;
}

typedef struct {
    uint16_t raw_temperature;
    uint16_t last_temperature;
}

```

```

    char sign;
    uint8_t integer_part;
    uint16_t decimal_part;
} temperature_t;

const uint8_t integer_lookup[7] = {1, 2, 4, 8, 16, 32, 64};
const uint16_t decimal_lookup[4] = {62, 125, 250, 500};

void format_temperature(temperature_t *t, uint16_t raw_temp) {

    /* find the sign, + or - */
    uint16_t masked_sign = raw_temp & 0xF800;           // get bin sign part
    t->sign = (masked_sign == 0xF800) ? '-' : '+';

    /* use absolute value for lookup */
    uint16_t abs_temp = raw_temp;
    if (t->sign == '-')
        abs_temp = (~raw_temp) + 1;           // invert bits, then add 1 (2's
        complement)

    /* extract integer & decimal part */
    uint8_t masked_int  = (abs_temp >> 4) & 0x7F; // get bin integer part
    uint8_t masked_dec  = abs_temp & 0x0F;        // get bin decimal part

    /* convert integer part using lookup table */
    uint16_t local_sum = 0;
    for (int i = 0; i < 7; i++) {
        if (masked_int & 0x01)
            local_sum += integer_lookup[i];
        masked_int >>= 1;
    }
    t->integer_part = local_sum;

    /* convert decimal part using lookup table */
    local_sum = 0;
    for (int i = 0; i < 4; i++) {
        if (masked_dec & 0x01)
            local_sum += decimal_lookup[i];
        masked_dec >>= 1;
    }
    t->decimal_part = local_sum;
}

```

IV. Keypad

```

/* USEFUL STRUCTURES FOR FUNCTIONALITY
*
* ROW_MASKS      -> mask for each individual row
* pressed_keys   -> current keyboard status

```

```

* KEY_ASCII    -> bit-to-ASCII lookup-table
*
*/
const uint8_t ROW_MASKS[4] = {0xFE, 0xFD, 0xFB, 0xF7};
uint16_t pressed_keys = 0;
const char KEY_ASCII[16] = {
    '*', '0', '#', 'D',    // Row 1, bits 0-3
    '7', '8', '9', 'C',    // Row 2, etc.
    '4', '5', '6', 'B',
    '1', '2', '3', 'A',
};
/*
 * This struct is designed to store both the full keypad state and the newly
 * pressed keys.
 * - "pressed_keys" keeps track of all keys currently pressed.
 * - "new_keys" tracks only the keys that were newly pressed since the last
 * scan.
 * This allows us to handle both continuous actions (holding a key)
 * and one-time events (new presses), which is useful for this and future
 * exercises.
 */
typedef struct {
    uint16_t all_pressed;
    uint16_t new_pressed;
} KeyStatus;

KeyStatus k = {0, 0};

/* --- SCAN ROW --- */
uint8_t scan_row(uint8_t row_num) {
    if (row_num < 1 || row_num > 4) return 0; // invalid row

    uint8_t row_mask = ROW_MASKS[row_num - 1];
    PCA9555_0_write(REG_OUTPUT_1, row_mask);

    uint8_t input = PCA9555_0_read(REG_INPUT_1);
    // return just column information (0 -> pressed key)
    return (input & 0xF0) >> 4;
}

/* --- SCAN KEYPAD ---
 *
 * Row 4: [Bit 12] [Bit 13] [Bit 14] [Bit 15] -> Col1 Col2 Col3 Col4
 * Row 3: [Bit 8 ] [Bit 9 ] [Bit 10] [Bit 11] -> Col1 Col2 Col3 Col4
 * Row 2: [Bit 4 ] [Bit 5 ] [Bit 6 ] [Bit 7 ] -> Col1 Col2 Col3 Col4
 * Row 1: [Bit 0 ] [Bit 1 ] [Bit 2 ] [Bit 3 ] -> Col1 Col2 Col3 Col4 (LSB)
 *
 */
uint16_t scan_keypad () {
    uint16_t state = 0;

```

```

    for (uint8_t row = 1; row <= 4; row++) {
        state |= ((uint16_t)scan_row(row)) << ((row - 1) * 4);
    }

    return ~state; // Inverse logic for easier manipulation (PRESSED ->
HIGH)
}

/* --- SCAN KEYPAD RISING EDGE --- */
KeyStatus scan_keypad_rising_edge () {
    uint16_t pressed_keys_tempo = 0;
    k.new_pressed = 0;

    pressed_keys_tempo = scan_keypad(); // Read current state

    _delay_ms(15); // Debouncing delay

    pressed_keys_tempo &= scan_keypad(); // Keep only stable pressed keys
(Low)

    k.new_pressed = pressed_keys_tempo & (~k.all_pressed); // Find newly
pressed keys

    k.all_pressed = pressed_keys_tempo; // Update pressed keys

    return k; // (PRESSED -> HIGH) logic
}

/* --- KEYPAD TO ASCII ---
*
* !!! ONLY ONE KEY PRESSED EACH TIME !!!
*
*/
KeyStatus keypad_to_ascii () {
    KeyStatus kk = scan_keypad_rising_edge();
    int flag[2] = {0};

    uint16_t new_keys = kk.new_pressed;
    uint16_t all_keys = kk.all_pressed;

    for (uint8_t bit = 0; bit < 16; bit++) {

        if (new_keys & (1 << bit)) { // if key is newly pressed
            kk.new_pressed = KEY_ASCII[bit]; // lookup ASCII + return
            flag[0] = 1;
        }

        if (all_keys & (1 << bit)) { // if key is newly pressed
            kk.all_pressed = KEY_ASCII[bit]; // lookup ASCII + return

```

```

        flag[1] = 1;
    }

}

if (flag[0] == 0) kk.new_pressed = '-';
if (flag[1] == 0) kk.all_pressed = '-';

return kk;
}

```

V. UART

```

/* Routine: usart_init
Description:
This routine initializes the
usart as shown below.
----- INITIALIZATIONS -----
Baud rate: 9600 (Fck= 8MH)
Asynchronous mode
Transmitter on
Reciever on
Communication parameters: 8 Data ,1 Stop, no Parity
-----
parameters: ubrr to control the BAUD.
return value: None.*/
void usart_init(unsigned int ubrr){
    UCSR0A=0;
    UCSR0B=(1<<RXEN0)|(1<<TXEN0);    // to receive + transmit
    UBR0H=(unsigned char)(ubrr>>8);
    UBR0L=(unsigned char)ubrr;
    UCSR0C=(3 << UCSZ00);    // 8-bit data
    return;
}

/* Routine: usart_transmit
Description:
This routine sends a byte of data
using usart.
parameters:
data: the byte to be transmitted
return value: None. */
void usart_transmit(uint8_t data){
    while(!(UCSR0A&(1<<UDRE0)));
    UDR0=data;
}

/* Routine: usart_receive
Description:

```

```

This routine receives a byte of data
from usart.
parameters: None.
return value: the received byte */
uint8_t usart_receive(){
    while(!(UCSR0A&(1<<RXC0)));
    return UDR0;
}

```

VI. ESP

```

/* Transmit entire command to uart */
void ESP_transmit_command(const char *cmd) {
    while(*cmd != '\0') {
        usart_transmit(*cmd);    // transmit byte after byte
        cmd++;
    }
}

/* Receive entire command from uart */
char* ESP_receive_response(){
    static char rsp[16];
    uint8_t inp;
    int i = 0;

    while(1) {
        inp = usart_receive();

        if (inp == '\n')
            break;
        else if (inp == '"')
            continue;
        else {
            if (i < 15) {
                rsp[i] = inp;
                i++;
            }
        }
    }

    rsp[i] = '\0';

    return rsp; // returns Success or Fail (without "", \n)
}

/* Transmit command, receive response & print on LCD accordingly */
void ESP_transceive_and_print_lcd(int n, const char *cmd, int server) {
    char *response;

```

```

while (1) {
    ESP_transmit_command(cmd);
    response = ESP_receive_response();

    if (!server) {
        if (strcmp(response, "Success") == 0 || strcmp(response, "Fail")
== 0) {
            print_lcd_with_number(n, response);
            break;
        }
        else {
            print_lcd_with_number(n, response);
            break;
        }
    }
}
}

```

Ζήτημα 1

Χρησιμοποιώντας τις απαραίτητες απο τις παραπάνω συναρτήσεις, υλοποιήσαμε τη main για το ζήτημα αυτό, με σκοπό την επιτυχή σύνδεση με τον server.

```

int main () {
    DDRC = 0x00;

    twi_init();
    PCA9555_0_write(REG_CONFIGURATION_0, 0x00); // EXT_PORT0 as output

    usart_init(103); // baud rate 9600
    lcd_init();

    lcd_clear_display();

    while (1) {

        /* Connect ESP to network */
        ESP_transceive_and_print_lcd(1, "ESP:connect\n", 0);
        _delay_ms(1000);

        /* Set URL accordingly */
        ESP_transceive_and_print_lcd(2,
"ESP:url:\n"http://192.168.1.250:5000/data\n", 0);
        _delay_ms(1000);

    }
}

```



```
    return 0;
}
```

Ζήτημα 2

Για την υλοποίηση του ζητήματος δημιουργήθηκαν οι ακόλουθες συναρτήσεις:

```
double getPatientsTemperature () {
    temperature_t t;    // create (t)emperature struct
    t.raw_temperature = receive_temperature();

    if (t.raw_temperature == 0x800)
        return -1;

    format_temperature(&t, t.raw_temperature); // pass by reference

    /* convert temp to double (ready for payload) */
    double temp = t.integer_part + t.decimal_part / 1000.0;
    temp += 12; // humanize room temp (~36oC)

    return temp; // assume positive (for human body)
}

void TempErrorChecking () {
    lcd_clear_display();
    lcd_command(0x80);
    lcd_string("Error:");
    lcd_command(0xC0);
    lcd_string("Thermometer");
    _delay_ms(1000);
    lcd_clear_display();
}

double getPatientsPressure () {
    int ADC_val = (int)ADC_conv(); // read POT1
    _delay_ms(100);
    double pressure = 20 * ADC_val / 1023.0; // (min, max) -> (0, 20 cm)

    if (pressure < 0.0 || pressure > 20.0)
        return -1.0;

    return pressure;
}

void PressErrorChecking () {
    lcd_clear_display();
    lcd_command(0x80);
    lcd_string("Error:");
    lcd_command(0xC0);
}
```

```

    lcd_string("BP Monitor");
    _delay_ms(1000);
    lcd_clear_display();
}

void determineStatus(double temp, double press, char *status, int flag) {
    if (flag) { // flag is set (1) when '0' is pressed
        strcpy(status, "NURSE CALL");
        return;
    }
    if (press < 4 || press > 12)
        strcpy(status, "CHECK PRESSURE");
    else if (temp < 34 || temp > 37)
        strcpy(status, "CHECK TEMP");
    else
        strcpy(status, "OK");
}

void printPatientMetrics(double temp, double press) {
    char buf[16];
    snprintf(buf, sizeof(buf), "%.2f", temp);
    lcd_command(0x80); // 1st line (temp, press)
    lcd_string("T:");
    lcd_string(buf);

    lcd_string(" P:");
    snprintf(buf, sizeof(buf), "%.2f", press);
    lcd_string(buf);
}

void printPatientStatus(char *status) {
    clear_2nd_line();
    lcd_command(0xC0); // 2nd line (status)
    lcd_string(status);
}

```

Η αντίστοιχη main του ζητήματος είναι:

```

int main () {
    DDRC = 0x00;
    DDRB = 0xFF;

    twi_init();
    PCA9555_0_write(REG_CONFIGURATION_0, 0x00); // IO as output to LCD

    usart_init(103); // baud rate 9600
    lcd_init();

    PWM_init();
}

```

```

ADC_init();

lcd_clear_display();

double temp;
double press;
char status[16];
char key;

/* MAIN loop */
while (1) {

    /* Connect ESP to network */
    ESP_transceive_and_print_lcd(1, "ESP:connect\n", 0);
    _delay_ms(500);

    /* Set URL accordingly */
    ESP_transceive_and_print_lcd(2,
"ESP:url:\\"http://192.168.1.250:5000/data\\"n", 0);
    _delay_ms(500);

    clear_2nd_line();

    PCA9555_0_write(REG_CONFIGURATION_1, 0xF0);    // IO1[0:3] as
output, IO1_[4:7] as input

    uint16_t i = 2000;
    while (i != 0) {

        /* TEMPERATURE loop */
        while (1) {
            temp = getPatientsTemperature();
            if (temp < 0) { TempErrorChecking(); continue; }
            else break;
        }

        /* PRESSURE loop */
        while (1) {
            press = getPatientsPressure();
            if (press < 0) { PressErrorChecking(); continue; }
            else break;
        }

        printPatientMetrics(temp, press);

        key = keypad_to_ascii().all_pressed;
        if (key != '-') {
            switch (key) {
                case '0': determineStatus(temp, press, status, 1);

```

```

break;

                case '#': determineStatus(temp, press, status, 0);
break;

                default: strcpy(status, "Invalid Key"); break;
            }
            printPatientStatus(status);
            _delay_ms(2000);

            break;
        }

        i--;
        _delay_ms(1);
    }

    clear_2nd_line();
}

return 0;
}

```

Ζήτημα 3

Δημιουργήθηκε επιπρόσθετα η ακόλουθη συνάρτηση, για την αποστολή του payload.

```

void send_payload (double temp, double press, const char *status) {
    static char payload[256];

    snprintf(payload, sizeof(payload), "ESP:payload:
[{"name\":\"team\", \"value\":\"60\"},
{"name\":\"temperature\", \"value\":\"%.2f\"},
{"name\":\"pressure\", \"value\":\"%.2f\"},
{"name\":\"status\", \"value\":\"%s\"}]\n", temp, press, status);

    ESP_transceive_and_print_lcd(3, payload, 0);
    //_delay_ms(500);

    ESP_transceive_and_print_lcd(4, "ESP:transmit\n" , 1);
    //_delay_ms(500);
}

```

Τελικά, η άσκηση ολοκληρώνεται με την main:

```

int main () {
    DDRC = 0x00;
    DDRB = 0xFF;

    twi_init();
}

```

```

PCA9555_0_write(REG_CONFIGURATION_0, 0x00); // IO as output to LCD

usart_init(103); // baud rate 9600
lcd_init();

PWM_init();
ADC_init();

lcd_clear_display();

double temp;
double press;
char status[16];
char key;

strcpy(status, "UNKNOWN");

/* MAIN loop */
while (1) {

    /* Connect ESP to network */
    ESP_transceive_and_print_lcd(1, "ESP:connect\n", 0);
    //_delay_ms(500);

    /* Set URL accordingly */
    ESP_transceive_and_print_lcd(2,
"ESP:url:\\"http://192.168.1.250:5000/data\\"n", 0);
    //_delay_ms(500);

    clear_2nd_line();

    PCA9555_0_write(REG_CONFIGURATION_1, 0xF0); // keypad config

    /* ----- Window for key ----- */
    uint16_t window = 10;

    while (window--) {

        /* TEMPERATURE loop */
        while (1) {
            temp = getPatientsTemperature();
            if (temp < 0) { TempErrorChecking(); continue; }
            break;
        }

        /* PRESSURE loop */
        while (1) {
            press = getPatientsPressure();
            if (press < 0) { PressErrorChecking(); continue; }
            break;
        }
    }
}

```

```

    }

    printPatientMetrics(temp, press);

    /* check keypad */
    key = keypad_to_ascii().all_pressed;
    if (key != '-') {
        switch (key) {
            case '0': determineStatus(temp, press, status, 1);
break;

            case '#': determineStatus(temp, press, status, 0);
break;

            default : strcpy(status, "Invalid Key"); break;
        }

        printPatientStatus(status);
        _delay_ms(1000);
        send_payload(temp, press, status);
        goto NEXT_CYCLE;    // go to next main loop
    }
}

send_payload(temp, press, status);

NEXT_CYCLE:
    clear_2nd_line();
}

return 0;
}

```